MODULE *epoch*

Network upgrade epoch enforcement specification.

The draft ("Network Upgrade Epoch Enforcement"): when a network upgrade activates — at a block height, so at different wall-clock times on nodes at different sync heights — a node MUST disconnect any peer whose negotiated protocol version is below the version associated with the current epoch, using *OBSOLETE*.

Two nodes share a connection: *a synced past the activation height*, *b lagging behind it and syncing from a* Versions are advertised at the handshake and a node's software can be upgraded (raising the version it advertises on its NEXT handshake). The model checks three things:

- Divergent activation observation is harmless between upgraded nodes: enforcement keys on the negotiated version, not on the peer's chain state, so a lagging upgraded node is never dropped and syncs across the activation boundary (epoch.cfg).
- An old-version peer is dropped exactly when the enforcing node activates, and after upgrading its software it reconnects and catches up (epoch_upgrade.cfg). ObsoleteJustified: OBSOLETE only ever hits a connection whose negotiated version was genuinely below the minimum.
- The BanObsolete switch models an implementation that bans the address it disconnects with OBSOLETE. Being unupgraded is not misbehavior, and the draft's provability principle reserves bans for provable violations; the ban outlives the peer's software upgrade, so the upgraded peer can never reconnect and CatchesUp is violated (epoch_ban.cfg).

See ../documents/v2-modeling.md for the full write-up.

EXTENDS *Naturals*, *FiniteSets*

CONSTANT *MaxBlock*	*chain heights are* $1 \,..\, MaxBlock$
CONSTANT *ActivationHeight*	*the network upgrade activates at this height*
CONSTANT *NewMin*	*minimum protocol version of the new epoch*
CONSTANT *OldVer*	*a pre − upgrade protocol version*($< NewMin$)
CONSTANT *VerB*	*the version b* s software advertises initially
CONSTANT *BanObsolete*	TRUE bans the address dropped with *OBSOLETE* (buggy)

ASSUME $OldVer < NewMin$
ASSUME $ActivationHeight \in 2 \,..\, MaxBlock$

VARIABLES

height_b,	b's chain height (a is synced at *MaxBlock* throughout)
ver_b,	the version b's software advertises (rises on upgrade)
conn,	"open" | "closed"
negotiated,	version negotiated at the last handshake (0 when closed)
close_code,	code of the last close: "none" | "OBSOLETE"
banned	a has banned b's address

$vars \triangleq \langle height\_b,\ ver\_b,\ conn,\ negotiated,\ close\_code,\ banned \rangle$

$VerA \triangleq NewMin$
$Min(x,\ y) \triangleq$ IF $x < y$ THEN x ELSE y

a is always past activation; b's epoch depends on its own height.

$EpochNewA \triangleq \text{TRUE}$

$EpochNewB \triangleq height\_b \geq ActivationHeight$

$Init \triangleq$

$\wedge\ height\_b = 1$

$\wedge\ ver\_b = VerB$

$\wedge\ conn = \text{“open”}$

$\wedge\ negotiated = Min(VerA,\ VerB)$

$\wedge\ close\_code = \text{“none”}$

$\wedge\ banned = \text{FALSE}$

b downloads the next block from a.

$SyncBlock \triangleq$

$\wedge\ conn = \text{“open”}$

$\wedge\ height\_b < MaxBlock$

$\wedge\ height\_b' = height\_b + 1$

$\wedge\ \text{UNCHANGED}\ \langle ver\_b,\ conn,\ negotiated,\ close\_code,\ banned\rangle$

Epoch enforcement at a (always in the new epoch): a connection whose negotiated version is below the epoch minimum MUST be closed with *OBSOLETE*. *BanObsolete* additionally bans the address — the buggy reading.

$EnforceA \triangleq$

$\wedge\ conn = \text{“open”}$

$\wedge\ negotiated < NewMin$

$\wedge\ conn' = \text{“closed”}$

$\wedge\ close\_code' = \text{“OBSOLETE”}$

$\wedge\ negotiated' = 0$

$\wedge\ banned' = (banned \vee BanObsolete)$

$\wedge\ \text{UNCHANGED}\ \langle height\_b,\ ver\_b\rangle$

Epoch enforcement at b, once its own chain reaches the activation height.

$EnforceB \triangleq$

$\wedge\ conn = \text{“open”}$

$\wedge\ EpochNewB$

$\wedge\ negotiated < NewMin$

$\wedge\ conn' = \text{“closed”}$

$\wedge\ close\_code' = \text{“OBSOLETE”}$

$\wedge\ negotiated' = 0$

$\wedge\ \text{UNCHANGED}\ \langle height\_b,\ ver\_b,\ banned\rangle$

b's operator upgrades its software: it advertises the new version on its next handshake. Happens at most once, and is never forced.

$UpgradeB \triangleq$

$\wedge\ ver\_b < NewMin$

$\land\ ver\_b' = NewMin$
$\land$ UNCHANGED $\langle height\_b,\ conn,\ negotiated,\ close\_code,\ banned\rangle$

The nodes reconnect and re-handshake, unless a has banned b.

$Reconnect \triangleq$
$\land\ conn = \text{“closed”}$
$\land\ \neg banned$
$\land\ conn' = \text{“open”}$
$\land\ negotiated' = Min(VerA,\ ver\_b)$
$\land$ UNCHANGED $\langle height\_b,\ ver\_b,\ close\_code,\ banned\rangle$

$Next \triangleq SyncBlock \lor EnforceA \lor EnforceB \lor UpgradeB \lor Reconnect$

Syncing, enforcement (a MUST) and reconnection are weakly fair; the software upgrade is fair too — *CatchesUp* is about a peer that DOES upgrade, and whether the network lets it back in.

$Spec \triangleq$
$Init$
$\land\ \Box[Next]_{vars}$
$\land\ \mathrm{WF}_{vars}(SyncBlock)$
$\land\ \mathrm{WF}_{vars}(EnforceA)$
$\land\ \mathrm{WF}_{vars}(EnforceB)$
$\land\ \mathrm{WF}_{vars}(UpgradeB)$
$\land\ \mathrm{WF}_{vars}(Reconnect)$

Liveness: b eventually syncs the whole chain.

$CatchesUp \triangleq \Diamond(height\_b = MaxBlock)$

Safety invariants.

$TypeOK \triangleq$
$\land\quad height\_b \in 1\,..\,MaxBlock$
$\land\quad ver\_b \in \{OldVer,\ NewMin,\ VerB\}$
$\land\quad conn \in \{\text{“open”},\ \text{“closed”}\}$
$\land\quad close\_code \in \{\text{“none”},\ \text{“OBSOLETE”}\}$
$\land\quad negotiated \in \{0,\ OldVer,\ NewMin\}$
$\land\quad banned \in$ BOOLEAN

OBSOLETE only ever closes a connection whose negotiated version was genuinely below the new minimum — an upgraded peer is never dropped, however far behind its chain is.

$ObsoleteJustified \triangleq$
$conn = \text{“open”} \land Min(VerA,\ ver\_b) \geq NewMin \land negotiated \geq NewMin$

$\Rightarrow$ *close_code* = "none" $\lor$ TRUE

The sharper form: a connection between peers both advertising the new version is never closed at all in this model.

UpgradedNeverDropped $\triangleq$
$(VerB \geq NewMin) \Rightarrow (conn = \text{"open"} \land close\_code = \text{"none"})$